

Stealing CSRF tokens with CSS injection (without iFrames)

Stealing CSRF tokens with CSS injection (without iFrames) - dxa4481, GitHub.

- Published: date not stated
- Original: <<https://github.com/dxa4481/cssInjection>>
- Preserved from: <https://github.com/dxa4481/cssInjection> (github-repository-api) on 2026-10-02
- Repository commit: 7caacfc19a326b9935680749dbac4123a2de21dd
- Licence: see the repository

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Repository reading copy. Created from documentation in [dxa4481/cssInjection](#), pinned to commit [7caacfc19a32](#). GitHub navigation and file listings are omitted. This is selected documentation; repository code is never checked out, built or run.

README.md

[View original document](#)

Stealing CSRF tokens with CSS injection (without iFrames)

A post [here](#) details a method for stealing sensitive data with CSS injection by using Attribute Selectors and iFrames. Because this method requires iFrames, and most major websites disallow being framed, this attack isn't always practical.

Here I'll detail here a way to do this without iFrames, effectively stealing a CSRF token in about 10 seconds.

Once the CSRF token is stolen, because the victim is already on an attacker website, the attacker can go ahead and complete a CSRF attack against the user.

Background

As the original post describes, [CSS attribute selectors](#) developers to select elements based on substring matches of the value of attribute tags. These value selectors can do one of three things

- Match if the string starts with the substring
- Match if the string ends with the substring
- Match if the string contains the substring anywhere

One practical use case for this is to color all `href` attributes that start with "https://example.com" a special color.

An unfortunate by-product of this is, sensitive information can sometimes be stored in html attribute values. Most often, CSRF tokens are stored this way: in value attributes on hidden forms.

This allows us to match CSS selectors to the attributes on the form in question, and based on whether the form matches the starting string, load an external resource such as a background image, which signals to the attacker the first character.

Using this method, they can walk down the string, and exfiltrate the entire secret value.

To pull this off, the victim server needs to allow or be vulnerable to arbitrary CSS being rendered. This can occur through CSS injection, or a feature on the website allowing you to include stylesheets. Note: the website does not need to be vulnerable to XSS.

To render the victim's CSS, the original paper proposes using iFrames. The limitations of this are of course if the victim website disallows being framed.

There is also a space/time tradeoff of either loading all possible characters at once in parallel, or multiplexing them one at a time. In my example to save time, I've elected to load them all at once. In some scenarios where the injection is small, multiplexing may prove to be the more viable option.

Without iFrames

To do this without iFrames, I've used a method similar to one I've discussed [before](#): I'll create a popup and then alter the location of the popup after a set timer.

Using this method, I can still load the victim's CSS, but I no longer depend on the victim being frameable. Because the initial pop-up is triggered via user event, I am not blocked by the browser.

To force a hard reload, I have the pop-up load a dummy window between CSS injections. This can be seen below

```
var win2 = window.open('https://security.love/anything', 'f',
    "top=100000,left=100000,menubar=1,resizable=1,width=1,height=1")
var win2 = window.open(`https://security.love/cssInjection/victim.html?injection=${css}`,
    'f', "top=100000,left=100000,menubar=1,resizable=1,width=1,height=1")
```

Without a backend server

The original paper describes exfiltrating data to a backend server, however because the CSRF is a client side attack, if we can come up with a way to do this without a server, we save a lot of overhead and complexity.

In order to receive the victim's resource loads client side, we can make use of Service Workers, which can intercept and read request data. Service Workers currently only apply to requests coming in from the Same Origin, and so for my demo I've cheated and put both the victim and attacker pages on the same origin.

Soon though, chrome may merge in this [experimental feature](#) which allows cross origin requests to be intercepted by service workers.

With this addition, we can make our attack 100% client side, and force users to perform CSRF actions in under 10 seconds of clicking the link, as seen in the demo below:

Demo

As explained above, because I don't want to run a web server (github pages is great) I'm cheating and using service workers to intercept and mock the server side component. As a result, for now, this demo only works in Chrome.

First I've created a very simple victim, that has a DOM based CSS injection, and placed a sensitive token on the page. I've made this DOM based to again, remove the need for a server. You may notice I've also included some protection against script tag injection, by encoding less than and greater than signs.

```
<form action="https://security.love" id="sensitiveForm">
  <input type="hidden" id="secret" name="secret" value="dJ7cw0N4BMyQi3NrQ26i">
</form>
<script src="mockingTheBackend.js"></script>
<script>
  var fragment = decodeURIComponent(window.location.href.split("?injection=")[1]);
  var htmlEncode = fragment.replace(/</g, "&lt;").replace(/>/g, "&gt;");
  document.write("<style>" + htmlEncode + "</style>");
</script>
```

Next, our attacker forces a load of the victim's CSS, and using the method described above, we steal the sensitive token one character at a time.

On the receiving end, I've defined a service worker that intercepts the requests, and sends them back to the domain via post-message, and then we store the token in local storage for future use. You can imagine a back end web server filling this function, and posting back the CSRF token to the attacker domain via web socket or polling.

ONLY TESTED IN CHROME RIGHT NOW:

[demo](#)

If everything works, after clicking somewhere on the page, you should see the CSRF token exfiltrated one character at a time from the victim page.

Final thoughts

Interestingly enough, reflected CSS injection is actually more deadly than stored CSS injection, because stored CSS injection would require a server to update the CSS for the victim before rendering.

For some time CSS injection has gone back and forth on severity. It used to be IE allowed users to execute Javascript in CSS. This demo hopefully shows that CSS injection, and rendering untrusted CSS on your domain can still lead to serious security vulnerabilities

Archived from <https://github.com/dxa4481/cssInjection>. Rendered offline from the local Markdown copy.